# Kommo Integration

## 1. Overview

### 1.1 Description

**Kommo Integration** connects **KommoCRM** (sales-focused CRM platform) with **Newo Platform**.

It enables:

- Automatic contact creation and updates in KommoCRM after each conversation
- Lead management with AI-powered pipeline stage selection
- Conversation transcript and analysis notes attached to CRM records
- Lead deduplication and tagging for tracking Newo-originated deals
- OAuth 2.0 authentication with automatic token refresh

This integration is designed for **sales teams and business owners** who need **automatic CRM synchronization based on conversational AI agent interactions** (voice or chat).

### 1.2 Use Cases

- When a conversation ends → Analyze results and create/update contact and lead in KommoCRM
- When a new caller is detected → Create contact with phone number and name in CRM
- When a returning caller contacts → Find existing contact, update notes, and adjust lead stage
- When a conversation is successful → Move lead to the appropriate pipeline stage based on AI analysis
- When a conversation fails → Move lead to the corresponding failure stage based on custom rules

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Contact Creation | ✅ | By phone number, first/last name |
| Contact Lookup | ✅ | Search by phone number |
| Contact Notes | ✅ | Full conversation analysis attached as note |
| Lead Creation | ✅ | With pipeline, stage, tag, and contact link |
| Lead Stage Update | ✅ | AI-powered stage selection via LLM (Gemini) |
| Lead Deduplication | ✅ | Filters by configurable tag (default: "newo") |
| Pipeline Discovery | ✅ | Auto-fetches available pipelines and stages |
| Custom Decision Rules | ✅ | Free-text business rules for stage selection |
| OAuth 2.0 Token Management | ✅ | Auto-refresh with retry logic (up to 3 attempts) |
| Public Integration Mode | ✅ | Pre-configured with Newo's OAuth app |
| Private Integration Mode | ✅ | Customer's own OAuth app credentials |
| Booking / Availability | ❌ | CRM-only integration, no scheduling |

## 2. Requirements

Before installation:

- Active **KommoCRM** account
- **OAuth 2.0 authorization** — either via Newo's public app or your own private OAuth app
- Kommo account **subdomain** (e.g., `mycompany` from `mycompany.kommo.com`)

## 3. How to Setup

### 3.1 Step 1 — Authorize in KommoCRM

**Public Integration (recommended):**

1. In Newo platform, set `kommo_subdomain` to your Kommo subdomain
2. Click the KommoCRM authorization link provided by the platform
3. Authorize the Newo app in your KommoCRM account
4. Select the sub-account (workspace) if prompted
5. Copy the **authorization code** from the redirect URL
6. Paste the code into the `kommo_oauth_code` attribute

**Private Integration:**

1. Log in to **KommoCRM → Settings → Integrations**
2. Create a new **private integration**
3. Copy the **Client ID** and **Client Secret**
4. Set `kommo_integration_type` to "Private Integration"
5. Enter your Client ID and Client Secret in the corresponding attributes
6. Complete the OAuth authorization and paste the code

### 3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `kommo_subdomain` | ✅ | Your Kommo account subdomain (e.g., `mycompany`) |
| `kommo_oauth_code` | ✅ | One-time OAuth authorization code |
| `kommo_integration_type` | ❌ | `Public Integration` or `Private Integration` (default: Private) |
| `kommo_client_id` | ❌ | OAuth Client ID (only for Private Integration) |
| `kommo_client_secret` | ❌ | OAuth Client Secret (only for Private Integration) |
| `kommo_enable_client_history` | ❌ | Enable CRM sync on session end (default: `False`) |
| `kommo_pipeline` | ❌ | Auto-populated from available pipelines |
| `kommo_pipeline_decision_rules` | ❌ | Custom business rules for AI stage selection |
| `kommo_lead_tag` | ❌ | Tag for Newo-created leads (default: `newo`) |

3. Click **Save + Publish All**

4. On publish, the SetupFlow automatically:
    - Exchanges the OAuth code for access and refresh tokens
    - Fetches available pipelines from KommoCRM
    - Fetches pipeline stages for the selected pipeline
    - Stores tokens and pipeline configuration

# 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

## 4.1 How the Integration Works

The integration operates based on **Triggers and Actions** via the event-driven system.

Unlike scheduling integrations, Kommo integration is **post-conversation** — it processes results after the conversation ends, not during it.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in KommoCRM

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Session Ended** | `session_ended` | When a conversation finishes — main trigger |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |
| **Test** | `test_integration` | Webhook test with sample conversation data |

**Available Actions**

- **Search Contact** — Find existing contact by phone number (GET `/contacts`)
- **Create Contact** — Create new contact with phone and name (POST `/contacts`)
- **Add Note** — Attach conversation analysis to contact (POST `/contacts/{id}/notes`)
- **Create Lead** — Create lead with pipeline stage and contact link (POST `/leads`)
- **Update Lead** — Change lead pipeline stage (PATCH `/leads/{id}`)
- **Get Lead** — Retrieve existing leads to check for deduplication (GET `/leads`)
- **Get Pipelines** — Discover available pipelines (GET `/leads/pipelines`)
- **Get Stages** — Fetch pipeline stages (GET `/leads/pipelines/{id}/statuses`)

## 4.2 Architecture Diagrams

**Overall Sequence**

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Kommo Integration
    participant T as Kommo OAuth
    participant API as KommoCRM API

    Note over I,T: Setup (on publish)
    A->>I: project_publish_finish
```

```
    I->>T: POST /oauth2/access_token (code)
    T-->>I: access_token + refresh_token
    I->>API: GET /leads/pipelines
    API-->>I: Pipelines list
    I->>API: GET /leads/pipelines/(id)/statuses
    API-->>I: Pipeline stages
    I->>A: Pipeline configured

    Note over U,API: Conversation happens
    U->>A: (conversation takes place)
    A->>U: (agent responds, conversation ends)

    Note over U,API: Post-Conversation CRM Sync
    A->>I: session_ended (summary, category, success, transcript, phone)
    I->>I: LLM analyze conversation -> select pipeline stage
    I->>API: GET /contacts?query=(phone)
    API-->>I: Contact results
    alt Contact not found
        I->>API: POST /contacts (name, phone)
        API-->>I: New contact_id
    end
    I->>API: POST /contacts/(id)/notes (analysis)
    API-->>I: Note created
    alt Existing lead with "newo" tag
        I->>API: PATCH /leads/(id) (status_id)
        API-->>I: Lead updated
    else No existing lead
        I->>API: POST /leads (pipeline, stage, contact, tag)
        API-->>I: Lead created
    end
```

**SalesFlow (Main Flow)**

```
flowchart TD
    E["session_ended"] --> GATE{"Client history<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| LLM["LLM analyze conversation:<br>stage_id, first_name,
<br>last_name, explanation"]
    LLM --> SEARCH["GET /contacts<br>?query={phone}"]
    SEARCH --> FOUND{"Contact<br>found?"}
    FOUND -->|"No"| CREATE["POST /contacts<br>{name, phone}"]
    CREATE --> NOTE
    FOUND -->|"Yes"| NOTE["POST /contacts/{id}/notes<br>{summary, transcript}"]
    NOTE --> LEADS{"Has existing<br>leads?"}
    LEADS -->|"No"| NEW_LEAD["POST /leads<br>{pipeline, stage,<br>contact,
tag:newo}"]
    LEADS -->|"Yes"| GET_LEAD["GET /leads<br>?filter[id]={ids}&with=tags"]
    GET_LEAD --> TAGGED{"Lead with<br>newo tag?"}
    TAGGED -->|"Yes"| UPDATE["PATCH /leads/{id}<br>{status_id}"]
    TAGGED -->|"No"| NEW_LEAD
```

```
    UPDATE --> DONE["Done"]
    NEW_LEAD --> DONE
```

**SetupFlow**

```
flowchart TD
    E["project_publish_finish"] --> INIT["Initialize attributes<br>and connectors"]
    INIT --> CODE{"OAuth code<br>provided?"}
    CODE -->|"No"| WAIT["Wait for code"]
    CODE -->|"Yes"| TOKEN["POST /oauth2/access_token<br>{code, client_id, secret}"]
    TOKEN --> OK1{"Tokens<br>received?"}
    OK1 -->|"No"| ERR["ResultError"]
    OK1 -->|"Yes"| SAVE["Save access_token<br>and refresh_token"]
    SAVE --> PIPE["GET /leads/pipelines"]
    PIPE --> SELECT["Auto-select pipeline<br>Create enum attribute"]
    SELECT --> STAGES["GET /leads/pipelines/{id}/statuses"]
    STAGES --> STORE["Store stages as JSON<br>for LLM decision-making"]
    STORE --> DONE["Setup complete"]
```

**Token Refresh (UtilsFlow)**

```
flowchart TD
    REQ["Any API request"] --> HTTP["Send via<br>kommo_connector"]
    HTTP --> STATUS{"HTTP<br>status?"}
    STATUS -->|"2xx"| SUCCESS["kommo_result_success"]
    STATUS -->|"401"| REFRESH["POST /oauth2/access_token<br>{grant_type:
refresh_token}"]
    REFRESH --> TOK_OK{"New token<br>received?"}
    TOK_OK -->|"Yes"| RETRY["Save tokens<br>Retry original request<br>(max 3
attempts)"]
    TOK_OK -->|"No"| ERR["kommo_result_error"]
    STATUS -->|"4xx/5xx"| ERR
    RETRY --> HTTP
```

### 4.3 Where to Test

Testing can be done by:

- Triggering a real conversation through the **Newo conversation interface** (chat or voice) and verifying CRM records after the session ends
- Using the `test_integration` webhook event with sample conversation data

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish the project and verify logs show successful OAuth token exchange and pipeline discovery
2. **CRM Sync**: Complete a conversation through the agent — after session ends, verify:
    - Contact appears in KommoCRM with correct phone and name
    - Conversation note is attached to the contact

    - Lead is created in the correct pipeline stage with "newo" tag
3. **Lead Update**: Complete a second conversation with the same phone number — verify:
    - Existing contact is found (not duplicated)
    - New note is added
    - Existing "newo"-tagged lead stage is updated (not duplicated)

**If no action occurs:**

- Ensure `kommo_enable_client_history` is set to `True`
- Verify OAuth tokens were obtained (check `kommo_access_token` is not empty)
- Confirm `kommo_subdomain` is correct
- Check that `kommo_pipeline` is selected
- Verify the OAuth code hasn't expired (codes are single-use)
- Re-publish the project if credentials were changed

> *Note: This integration performs real operations in KommoCRM. Contacts and leads created through the agent are reflected in the live CRM system.*

## 5. FAQ

**Q: Does the integration import historical CRM data?** A: No. The integration only creates/updates contacts and leads based on conversations that happen after activation.

**Q: Can I connect multiple KommoCRM accounts?** A: Each integration instance supports one Kommo account (subdomain). For multiple accounts, create separate integration instances.

**Q: How does the AI select the pipeline stage?** A: After each conversation, an LLM (Gemini) analyzes the conversation summary, category, success status, and transcript. It applies custom business rules from `kommo_pipeline_decision_rules` and selects the most appropriate stage. The decision includes an explanation for transparency.

**Q: What is the difference between Public and Private integration?** A: **Public Integration** uses Newo's pre-registered OAuth app in KommoCRM — no additional setup needed. **Private Integration** lets you use your own OAuth app credentials, giving you more control over permissions and branding.

**Q: How are duplicate leads prevented?** A: Leads created by Newo are tagged with a configurable tag (default: "newo"). When a returning caller is detected, the integration searches for existing leads with this tag. If found, the lead's stage is updated. If not, a new lead is created.

**Q: What data is stored in the contact note?** A: Each note contains the full conversation analysis: summary, category, success/failure status, detailed results, and the complete transcript. This provides full context for sales team review.

**Q: What happens if the OAuth token expires?** A: The integration automatically detects HTTP 401 responses and refreshes the token using the refresh token. It retries the failed request up to 3 times. Token refresh uses a separate HTTP connector for isolation.